IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

ETAPA B · CONSTRUIR · SPRINT 6 DE 17

Sprint 6 · Datos listos para modelar

¿están los datos en condiciones de entrenar algo fiable?

FECHAS
3–5 de noviembre

HORAS
11 h (4 h teoría · 4 h práctica guiada · 3 h PBL)

CONTENIDOS
M2 (rol del Data Scientist, pandas, visualización, limpieza y preprocesado) y M3 UD 3.1–3.2 (flujo con scikit-learn, pipelines, métricas y validación).

Qué aprenderás

1. Explorar una tabla con pandas y elegir el gráfico adecuado para cada tipo de dato.
2. Encontrar defectos en un dataset (nulos, duplicados, *outliers*, tipos y fechas mal formados, categorías inconsistentes) y **razonar de dónde vienen**.
3. Construir un **pipeline** reproducible (`ColumnTransformer` + `Pipeline`) que impute, escale y codifique sin fuga de datos.
4. Medir un modelo con honestidad: *baseline*, métricas, validación cruzada, partición temporal.
5. Reconocer y evitar la **fuga de datos** (*data leakage*).

Requisitos previos

- Sprint 4 y 5: Python básico, NumPy y algo de estadística con pandas.
- Cuenta de Google (para Colab) o el entorno local del curso (Python 3.11).
- Datos: `datos/reservas_turisdata.csv` (limpio) y `datos/reservas_sucio.csv` (el del reto). Los notebooks los descargan solos si no los encuentran.

Prácticas

Notebook	Tema	Tiempo	Colab
`S06_01_pandas_eda`	pandas, EDA y visualización exploratoria; primer chequeo de calidad	60 min	Abrir en Colab
`S06_02_limpieza`	Limpieza: duplicados, tipos, fechas, categorías, imposibles, nulos	70 min	Abrir en Colab
`S06_03_preprocesado_pipeline`	Escalado, codificación, `ColumnTransformer` y `Pipeline`	55 min	Abrir en Colab
`S06_04_metricas_validacion`	Baseline, métricas, sobreajuste, validación cruzada, fuga y partición temporal	65 min	Abrir en Colab

Cada notebook tiene ejercicios con `assert`: si la celda de comprobación no da error, vas bien. Si terminas antes, dedica tiempo a las preguntas de reflexión del final de cada notebook.

RETO Reto PBL · «Datos sucios, cliente impaciente» (3 h, en equipo)

Contexto del cliente. TurisData Canarias nos manda por correo un fichero exportado a mano desde varios sistemas: `reservas_sucio.csv`. Quieren un primer modelo de cancelaciones "para ayer", pero el gerente no se fía de los

datos. Nos pide **un informe corto que le diga qué está mal y por qué**, y **un proceso que deje los datos listos cada vez que le llegue un fichero así**, con un modelo de referencia (*baseline*).

Qué se entrega (un repositorio o carpeta con):

1. `informe_calidad.md` (o PDF), 1–2 páginas: tabla de defectos (qué, dónde, cuántas filas, causa probable, tratamiento y su justificación) y las preguntas que haríais al cliente. Plantilla en `guias/S06/informe_calidad_plantilla.md`.
2. `reto_S06.ipynb`: notebook con el EDA, la función de limpieza y el **pipeline** de preprocesado + un modelo baseline evaluado con validación cruzada.
3. `README.md` con instrucciones para reproducirlo (versiones de librerías, cómo ejecutar).

Restricciones.

- Solo librerías del curso (pandas, scikit-learn, matplotlib, seaborn…).
- Nada de editar el CSV a mano: toda corrección debe estar en código.
- Todo estadístico aprendido de los datos (medianas, medias, categorías) debe ajustarse **solo con train**.
- No se descarta ninguna fila sin justificarlo en el informe.

Criterios de evaluación (0–4 cada uno; misma rúbrica de todos los retos):

Criterio	1 · Insuficiente	2 · Suficiente	3 · Bueno	4 · Excelente
Corrección técnica	Errores que invalidan el resultado	Correcto con fallos menores	Correcto y bien justificado	Correcto, justificado y con alternativas comparadas
Reproducibilidad	No se puede volver a ejecutar	Se ejecuta con ayuda	Se ejecuta desde el README	Además incluye datos o script y versiones fijas
Análisis y comunicación al cliente	Solo código o cifras sin interpretar	Interpretación básica	Conclusiones claras para el cliente	Recomendación accionable con límites explicados
IA responsable	No se considera	Menciones genéricas	Riesgos específicos identificados	Medidas aplicadas y evaluadas

Pista de IA responsable: ¿qué pasa si el país de origen se imputa mal? ¿Podría el modelo discriminar por nacionalidad? ¿Hay columnas que no deberían usarse?

Pasos sugeridos.

- **Día 1 (3/11), tras las prácticas 1–2:** *planning* (30 min), reparto de roles (Scrum Master y responsable técnico rotan), inventario de defectos con `resumen_calidad` y gráficos. Primer borrador de la tabla del informe.
- **Día 2 (4/11):** función `limpiar_reservas`, comprobación antes/después y separación train/test. Pipeline de preprocesado (práctica 3) y baseline con validación cruzada (práctica 4).
- **Día 3 (5/11):** cierre del informe, README, autoevaluación y coevaluación; ensayo de 5 minutos; **review + retrospectiva** (última hora).

Qué se enseña en la review (5 min por equipo): la tabla de defectos (los 3 más graves y por qué), el pipeline en marcha desde cero (`Restart & Run all`) y la métrica del baseline frente a un modelo "tonto".

Preguntas de la retrospectiva: ¿qué defecto nos costó más encontrar? ¿Qué decisión de limpieza discutimos más? ¿Qué haríamos distinto con el siguiente fichero del cliente?

Definition of Done

- [ ] El informe de calidad recoge todos los defectos encontrados con causa probable y tratamiento.
- [ ] El notebook se ejecuta de principio a fin sin errores (`Restart & Run all`).

- [ ] El pipeline ajusta imputación, escalado y codificación solo con train.
- [ ] Hay un baseline (`DummyClassifier`) y un modelo, comparados con validación cruzada.
- [ ] README con instrucciones y versiones de las librerías.
- [ ] Subido a Git; autoevaluación y coevaluación rellenadas.

Recursos

- Documentación de scikit-learn: *Pipelines and composite estimators* y *Cross-validation*.
- Documentación de pandas: *Working with missing data* y `pd.to_datetime`.
- Guía de usuario de scikit-learn: *Common pitfalls and recommended practices* (fuga de datos).
- Galería de seaborn (elección de gráficos).
- Tu propio `resumen_calidad` de la práctica 1: reutilízalo.